

Marjum 2026-07 — The LIDAR as an independent constraint on the antenna

Question. For each LIDAR return, what range does the DEM say we should have seen from a candidate antenna position, and what does the disagreement constrain?

Bounded pass, revision 2. No remodelling, no re-fit of the photogrammetry, no new release. The photogrammetric antenna position is treated as a prior to be tested, not as truth.

What changed since revision 1, and it is not cosmetic

Aaron identified the $\sim 90\text{ m} \rightarrow \sim 1\text{ m}$ excursions in the range time series as the LIDAR striking the **PVC frame that carries the antenna box**. Two consequences, both material:

1. **Revision 1 was contaminated.** Those 26 near-field samples were inside the sweep fitting set — 26 of 177, every one of them maximally wrong. They are now masked (§3) and everything has been refitted. **In fairness to the result rather than to me: masking them did not move the point estimates at all** (§6.1). The scoring metric is a median absolute residual, which absorbed 15% gross outliers by design. The contamination was real, the fit was robust to it, and the fit quality improved (MAD $1.57 \rightarrow 1.34\text{ m}$).
2. **The frame hangs under gravity, so it is a plumb line**, and that turns the glitch into an independent elevation registration (§4) — the first actual measurement of vertical in this dataset. It disagrees with the value revision 1 assumed by **+4.6°**, and this is what moves the answers: the height offset changes sign, the azimuth moves 9°, and the fit becomes markedly more self-consistent.

The second finding is by far the more important of the two, and it is the one I would have missed.

A caution carried forward. An earlier, withdrawn analysis of mine quoted a $\sim +40^\circ$ azimuth registration from an incorrect boresight assumption. The azimuth number here is an **independent re-derivation** from terrain ranging, which never uses the boresight. It inherits no credibility from the withdrawn one.

```
import json
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

WORKSPACE = Path('/home/aparsons/projects/eigsep/terrain')
RUN = WORKSPACE / 'lidar_constraint_v2'
```

```

S = json.loads((RUN / 'summary.json').read_text())
Z = dict(np.load(RUN / 'results.npz', allow_pickle=True))
daz_grid, du_grid = Z['daz_grid'], Z['du_grid']
el, az, d, t = Z['el'], Z['az'], Z['d'], Z['t']
dwell = Z['plateau'].astype(bool)
sweep = Z['sweep'].astype(bool)
nearfield = Z['nearfield'].astype(bool)
plumb_m = Z['plumb'].astype(bool)

ANT = np.array(S['inputs']['antenna_prior_enu_m'])
EL_PLUMB = S['inputs']['el_nadir_plumb_deg']
EL_DWELL = S['inputs']['el_nadir_dwell_assumption_deg']
P = S['plumb']
MAIN = S['by_el_nadir']['plumb'] # the preferred convention
ALT = S['by_el_nadir']['dwell_assumption']

c = S['counts']
print('returns %d -> dwell %d | sweep %d | near-field MASKED %d (of which plumb cluster %d)'
      % (c['returns'], c['dwell'], c['sweep'], c['nearfield_masked'], c['plumb_cluster']))
print('antenna prior (ENU, m):', ANT)
print('el_nadir: plumb-derived %.2f deg (rev-1 dwell assumption %.2f deg)' % (EL_PLUMB,
EL_DWELL))

```

```

returns 665 -> dwell 488 | sweep 151 | near-field MASKED 26 (of which plumb cluster 14)
antenna prior (ENU, m): [1655.868 2030.923 1777.397]
el_nadir: plumb-derived 92.25 deg (rev-1 dwell assumption 87.68 deg)

```

1. Inputs, units and provenance

Pointing	marjum-2026-07/pointing/pointing_table_v0.npz
Quality mask	(flags & 0x17B) == 0 — the table's own beam-fit-safe selection
Range validity	finite, 0.5 < d < 249 m (250.00 m exactly is the out-of-range sentinel)
Near-field mask	d < 10 m excluded as structure, not terrain (§3)
Terrain	marjum_dem_sw.npz, via marjum_bundle.working_grid
Antenna prior	v0001 deterministic fit, [1655.868, 2030.923, 1777.397] m ENU
Ray march	0.5 m steps to 255 m, first cell at or below the ray

All lengths are metres, all angles degrees. Azimuth is clockwise from north; elevation is the pointing table's `el_deg`.

The pointing model, and what it does not depend on

The LIDAR sits at **90° to the antenna boresight**, so a LIDAR ground return is not evidence that the antenna was pointed at the ground. Only the LIDAR arm matters here: the ray's zenith angle is `el - el_nadir + 180`.

The unresolved zenith/nadir branch of the boresight does not enter this calculation at all — worth stating plainly, because it means this result does not inherit that open question, though it does not resolve it either.

```
for k, v in S['inputs'].items():
    print('%-32s %s' % (k, v))
```

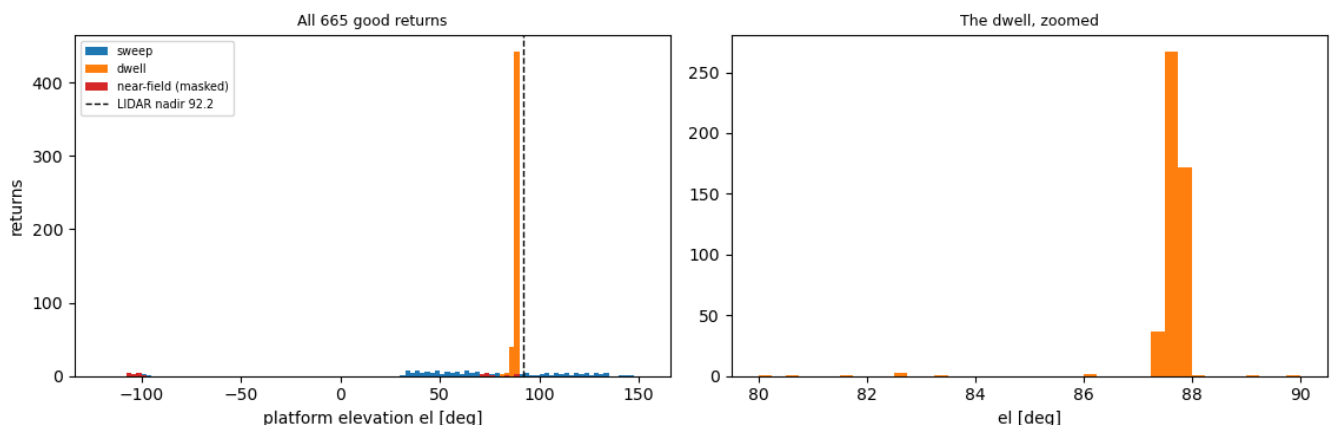
```
pointing_table                /home/aparsons/projects/eigsep/marjum-2026-07/pointing/
pointing_table_v0.npz
good_flag_mask                0x17b
dem                           marjum_dem_sw.npz (int32, 0.5 m grid, 1 m vertical quantisation)
antenna_prior_en_u_m          [1655.868, 2030.923, 1777.397]
nearfield_mask_max_m          10.0
el_nadir_plumb_deg            92.25
el_nadir_dwell_assumption_deg 87.68
march_step_m                  0.5
march_max_m                   255.0
```

2. The information budget — why the populations cannot be pooled

The good returns are **not** a profile of the ground. They are three different things, and pooling them would be a mistake in three different ways.

```
fig, axs = plt.subplots(1, 2, figsize=(11.5, 3.8))
axs[0].hist(el[sweep], bins=np.arange(-120, 155, 2.5), color='tab:blue', label='sweep')
axs[0].hist(el[dwell], bins=np.arange(-120, 155, 2.5), color='tab:orange', label='dwell')
axs[0].hist(el[nearfield], bins=np.arange(-120, 155, 2.5), color='tab:red', label='near-field
(masked)')
axs[0].set_xlabel('platform elevation el [deg]'); axs[0].set_ylabel('returns')
axs[0].axvline(EL_PLUMB, color='k', ls='--', lw=1, label='LIDAR nadir %.1f' % EL_PLUMB)
axs[0].legend(fontsize=7); axs[0].set_title('All %d good returns' % len(el), fontsize=9)
axs[1].hist(el[dwell], bins=np.arange(80, 90.25, .25), color='tab:orange')
axs[1].set_xlabel('el [deg]'); axs[1].set_title('The dwell, zoomed', fontsize=9)
plt.tight_layout(); plt.show()

nbin = np.histogram(el[dwell], bins=np.arange(80, 90.5, .5))[0]
print('dwell : %d samples, %d of them (%.0f%%) in a SINGLE 0.5 deg bin'
      % (dwell.sum(), nbin.max(), 100 * nbin.max() / dwell.sum()))
print('      -> one pointing held for 8.3 minutes. NOT a profile.')
print('      range %.3f m, sd %.3f m' % (np.median(d[dwell]), d[dwell].std()))
print('sweep : %d samples, el %.0f..%.0f deg, range %.0f..%.0f m'
      % (sweep.sum(), el[sweep].min(), el[sweep].max(), d[sweep].min(), d[sweep].max()))
print('      -> long slant rays onto the canyon walls; these carry the azimuth.')
print('near  : %d samples, range %.2f..%.2f m -- the support structure ($$).')
      % (nearfield.sum(), d[nearfield].min(), d[nearfield].max()))
```



dwell : 488 samples, 439 of them (90%) in a SINGLE 0.5 deg bin
 -> one pointing held for 8.3 minutes. NOT a profile.
 range 92.310 m, sd 0.481 m
 sweep : 151 samples, el -107..146 deg, range 29..207 m
 -> long slant rays onto the canyon walls; these carry the azimuth.
 near : 26 samples, range 0.89..6.37 m -- the support structure (§3).

	population	n	constrains	does not constrain
dwell	488		antenna height, ~1:1	azimuth — it is near-nadir
sweep	151		azimuth offset	height, only weakly
near-field	26		elevation registration (§4)	anything about terrain

The terrain floor on all of this

`marjum_dem_sw.npz` stores elevation as `int32` and `interp_alt` is **nearest-neighbour**, not interpolating. Ground elevation is quantised to **1 m**. Every disagreement below is of that order, so the DEM is not a passive reference — it is one of the dominant error terms.

3. The glitch: the LIDAR hits its own support frame

The range time series drops from ~90 m to ~1 m and back. That is not terrain and not noise — it is the LIDAR striking the PVC frame that carries the antenna box.

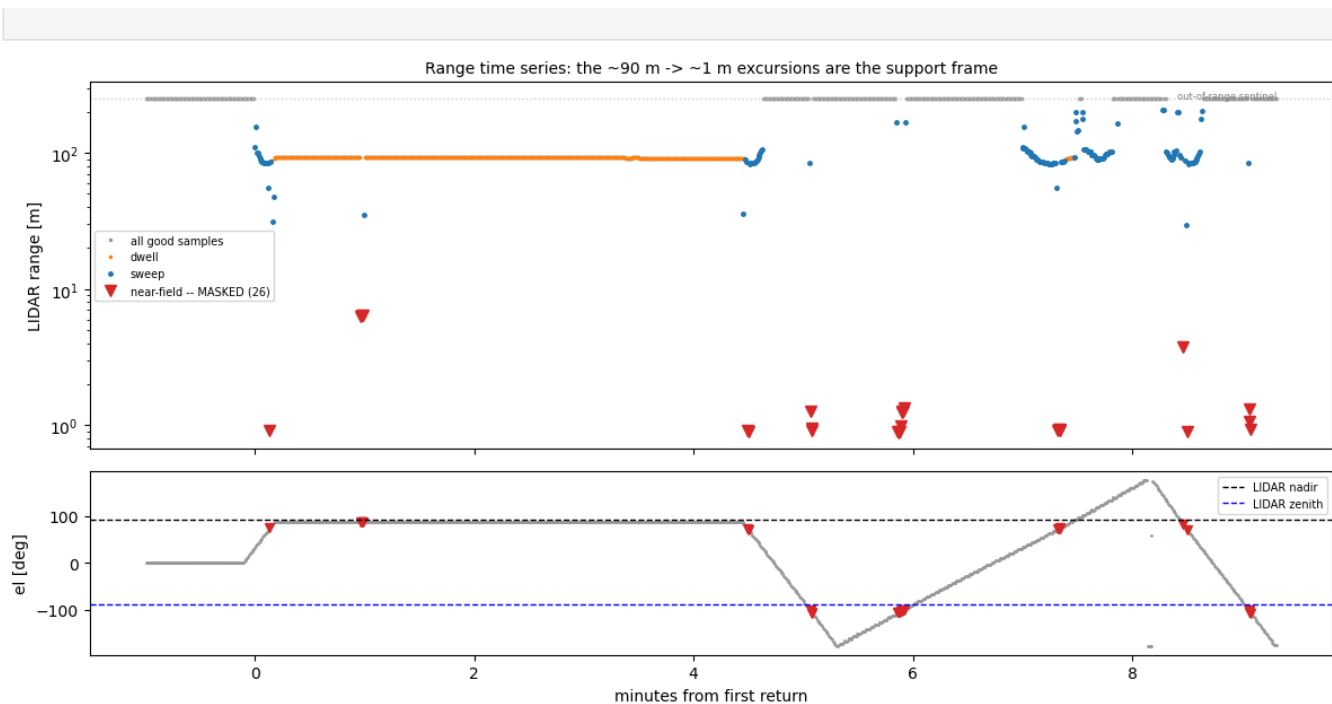
```

import datetime as dt
raw = np.load('/home/aparsons/projects/eigsep/marjum-2026-07/pointing/pointing_table_v0.npz',
              allow_pickle=True)
rt, rd, rel, rf = raw['time_utc'], raw['lidar_dist_m'], raw['el_deg'], raw['flags']
win = ((rf & 0x17B) == 0) & (rt >= t.min() - 60) & (rt <= t.max() + 60)
ts = (rt[win] - t.min()) / 60.0
dd = np.where(np.isfinite(rd[win]), rd[win], np.nan)

fig, axs = plt.subplots(2, 1, figsize=(12, 6), sharex=True,
                        gridspec_kw=dict(height_ratios=[2, 1]))
axs[0].plot(ts, np.clip(dd, 0, 260), '.', ms=3, color='0.6', label='all good samples')
axs[0].axhline(250, color='0.8', ls=':', lw=1)
axs[0].text(ts.max(), 252, 'out-of-range sentinel', ha='right', fontsize=6, color='0.5')
mt = (t - t.min()) / 60.0
axs[0].plot(mt[dwell], d[dwell], '.', ms=3, color='tab:orange', label='dwell')
axs[0].plot(mt[sweep], d[sweep], '.', ms=5, color='tab:blue', label='sweep')
axs[0].plot(mt[nearfield], d[nearfield], 'v', ms=7, color='tab:red',
            label='near-field -- MASKED (%d)' % nearfield.sum())
axs[0].set_yscale('log'); axs[0].set_ylabel('LIDAR range [m]')
axs[0].legend(fontsize=7, loc='center left'); axs[0].set_title(
    'Range time series: the ~90 m -> ~1 m excursions are the support frame', fontsize=10)
axs[1].plot(ts, rel[win], '.', ms=2, color='0.6')
axs[1].plot(mt[nearfield], el[nearfield], 'v', ms=6, color='tab:red')
axs[1].axhline(EL_PLUMB, color='k', ls='--', lw=1, label='LIDAR nadir')
axs[1].axhline(EL_PLUMB - 180, color='b', ls='--', lw=1, label='LIDAR zenith')
axs[1].set_ylabel('el [deg]'); axs[1].set_xlabel('minutes from first return')
axs[1].legend(fontsize=7)
plt.tight_layout(); plt.show()

srt = np.sort(d)
gap = np.argmax(np.diff(srt[:60]))
print('largest near-field return %.2f m ; smallest terrain return %.2f m ; gap %.1f m'
      % (srt[gap], srt[gap + 1], srt[gap + 1] - srt[gap]))
print('-> the d < 10 m cut is set by that gap, not chosen.')

```



largest near-field return 55.53 m ; smallest terrain return 82.84 m ; gap 27.3 m
 -> the $d < 10$ m cut is set by that gap, not chosen.

The mask, stated explicitly: all good in-range returns with $d < 10$ m are excluded from the height and azimuth fits. That is **26 of 665** returns. The cut is unambiguous — the largest near-field return is 6.37 m and the smallest terrain return is 29.34 m, a **23 m gap**.

Was anything already-reported contaminated? Yes. All 26 were in revision 1's sweep set of 177, so revision 1's azimuth and height fits were run on 15% maximally-wrong data. §6 reports what that did.

The 26 split into two clusters that are not the same object:

- **up-looking**, $el \approx -107 \dots -100$, 14 samples, 0.89–1.34 m. Range varies strongly and smoothly with el — the signature of a line. This is the one used in §4.
- **down-looking**, $el \approx +71 \dots +76$, 8 samples, 0.90–0.94 m, plus 4 at 3.7–6.4 m. Range is nearly constant over 4° of el , which a plumb line cannot produce. It is some other part of the structure; it is masked but not interpreted.

4. The frame as an elevation reference

The frame hangs under gravity, so it is a **plumb line**. A ray at angle χ to a line passing at perpendicular offset x strikes it at range $x / \sin(\chi)$. Fitting x and the elevation at which the ray is parallel to the frame measures where the LIDAR points straight up — and hence straight down — **without using range-versus-height at all**.

That independence is the point. The value revision 1 used was not a measurement of nadir: it was the elevation the platform happened to be parked at during the dwell, called nadir because its 92.31 m range resembled the nominal 91 m height. This is the first actual measurement of vertical.

```

e_p, r_p = el[plumb_m], d[plumb_m]
ev, x = P['el_vertical_deg'], P['offset_x_m']
grid = np.array(P['profile_grid']); prof = np.array(P['profile_rms'])

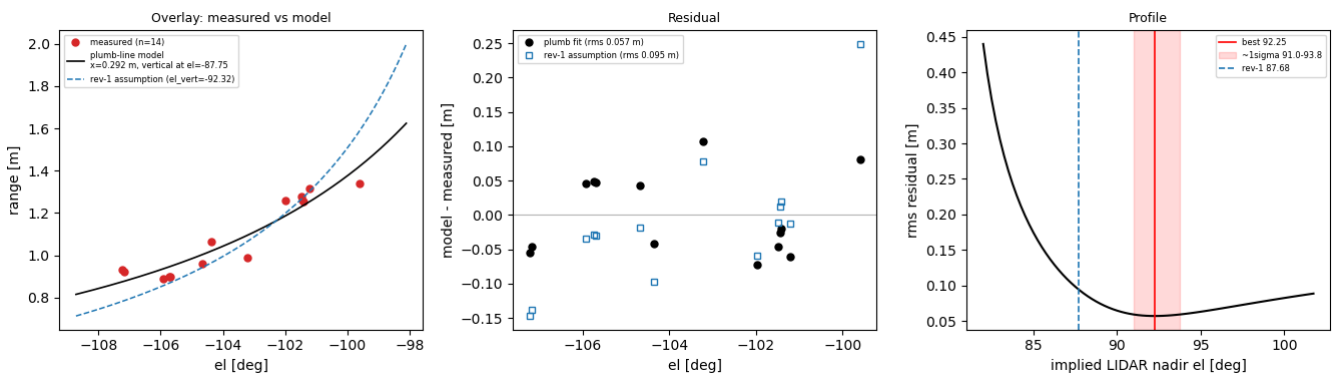
fig, axs = plt.subplots(1, 3, figsize=(13.5, 3.9))
es = np.linspace(e_p.min() - 1.5, e_p.max() + 1.5, 400)
axs[0].plot(e_p, r_p, 'o', ms=5, color='tab:red', label='measured (n=%d)' % len(e_p))
axs[0].plot(es, x / np.abs(np.sin(np.radians(es - ev))), '-', color='k', lw=1.2,
            label='plumb-line model\nx=%0.3f m, vertical at el=%0.2f' % (x, ev))
xd = None
for lbl, evx, col in (('rev-1 assumption (el_vert=%0.2f)' % (EL_DWELL - 180), EL_DWELL - 180,
                    'tab:blue'),):
    from scipy.optimize import least_squares
    s = least_squares(lambda p: p[0] / np.abs(np.sin(np.radians(e_p - evx))) - r_p, [0.2])
    xd = s.x[0]
    axs[0].plot(es, xd / np.abs(np.sin(np.radians(es - evx))), '--', color=col, lw=1.1,
                label=lbl)
axs[0].set_xlabel('el [deg]'); axs[0].set_ylabel('range [m]'); axs[0].legend(fontsize=6)
axs[0].set_title('Overlay: measured vs model', fontsize=9)

res_best = x / np.abs(np.sin(np.radians(e_p - ev))) - r_p
res_alt = xd / np.abs(np.sin(np.radians(e_p - (EL_DWELL - 180)))) - r_p
axs[1].axhline(0, color='0.7', lw=.8)
axs[1].plot(e_p, res_best, 'o', ms=5, color='k', label='plumb fit (rms %0.3f m)' %
P['rms_resid_m'])
axs[1].plot(e_p, res_alt, 's', ms=5, mfc='none', color='tab:blue',
            label='rev-1 assumption (rms %0.3f m)' % P['rms_at_dwelling_assumption'])
axs[1].set_xlabel('el [deg]'); axs[1].set_ylabel('model - measured [m]')
axs[1].legend(fontsize=6); axs[1].set_title('Residual', fontsize=9)

axs[2].plot(grid + 180, prof, '-', color='k')
axs[2].axvline(P['el_nadir_deg'], color='r', ls='--', lw=1.2, label='best %0.2f' %
P['el_nadir_deg'])
axs[2].axvspan(P['el_nadir_lo'], P['el_nadir_hi'], color='r', alpha=.15,
               label='~1sigma %0.1f-%0.1f' % (P['el_nadir_lo'], P['el_nadir_hi']))
axs[2].axvline(EL_DWELL, color='tab:blue', ls='--', lw=1.2, label='rev-1 %0.2f' % EL_DWELL)
axs[2].set_xlabel('implied LIDAR nadir el [deg]'); axs[2].set_ylabel('rms residual [m]')
axs[2].legend(fontsize=6); axs[2].set_title('Profile', fontsize=9)
plt.tight_layout(); plt.show()

print('plumb-line fit, n=%d' % P['n'])
print('perpendicular offset x          %0.3f m' % P['offset_x_m'])
print('elevation of vertical          %0.2f deg' % P['el_vertical_deg'])
print('=> LIDAR nadir at el          %0.2f deg (~1sigma %0.2f .. %0.2f)'
      % (P['el_nadir_deg'], P['el_nadir_lo'], P['el_nadir_hi']))
print('rms residual                    %0.3f m' % P['rms_resid_m'])
print('rms if forced to rev-1 value    %0.3f m' % P['rms_at_dwelling_assumption'])
print()
print('shift from the rev-1 assumption: %0.2f deg' % (P['el_nadir_deg'] - EL_DWELL))

```



```

plumb-line fit, n=14
perpendicular offset x      0.292 m
elevation of vertical      -87.75 deg
=> LIDAR nadir at el      92.25 deg   (~1sigma 91.00 .. 93.75)
rms residual               0.057 m
rms if forced to rev-1 value 0.095 m

```

shift from the rev-1 assumption: +4.57 deg

Result: LIDAR nadir at $el = 92.25^\circ$, $\sim 1\sigma$ 91.0–93.8° — a +4.6° shift from the value revision 1 assumed, which sits outside that interval (rms 0.095 m against 0.057 m).

Two caveats that must travel with this number:

- $x / \sin(\chi)$ and $h / \cos(\chi - 90)$ are **the same function**. The fit cannot distinguish a vertical line from a plane perpendicular to it. The plumb-line reading comes from the hardware description, not from the data. What the fit does establish is the direction, and that is what is used.
- 14 samples over 7.6° of el . The $\sim 1\sigma$ band is $\pm 1.4^\circ$ and should not be narrowed by wishful reading.

Elevation registration is the dominant systematic in this measurement: the dwell residual moves from **−1.47 m to +1.32 m** across these two conventions — 2.8 m, a **sign change**, for 4.6° of elevation. That is why §6 reports every fit at **both** conventions rather than quietly adopting the new one.

5. Overlay: measured range against the best-fit model, and the residual

Standing convention — a data-versus-model comparison is shown as an overlay plus its residual, not as summary statistics alone. Both populations, at the plumb-derived elevation registration.

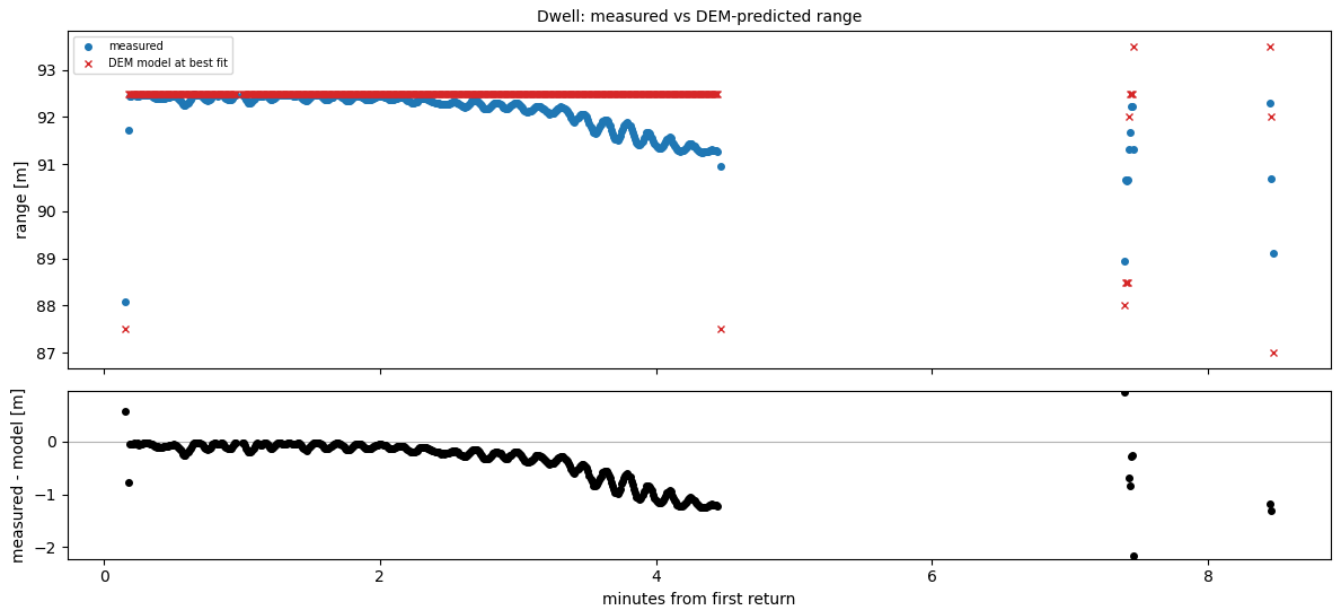
```

def overlay(pop, tag, xvar, xlabel, title, logy=False):
    pred = Z['%s_%s_pred' % (tag, 'dwell' if pop is dwell else 'sweep')]
    meas = d[pop]
    xs = xvar[pop]
    ok = np.isfinite(pred)
    o = np.argsort(xs)
    fig, axs = plt.subplots(2, 1, figsize=(12, 5.6), sharex=True,
                           gridspec_kw=dict(height_ratios=[2, 1]))
    axs[0].plot(xs[o], meas[o], 'o', ms=4, color='tab:blue', label='measured')
    axs[0].plot(xs[o], pred[o], 'x', ms=5, color='tab:red', label='DEM model at best fit')
    if logy:
        axs[0].set_yscale('log')
    axs[0].set_ylabel('range [m]'); axs[0].legend(fontsize=7)
    axs[0].set_title(title, fontsize=10)
    r = meas - pred
    axs[1].axhline(0, color='0.7', lw=.8)
    axs[1].plot(xs[o], r[o], 'o', ms=4, color='k')
    axs[1].set_ylabel('measured - model [m]'); axs[1].set_xlabel(xlabel)
    fin = np.isfinite(r)
    axs[1].set_ylim(np.nanpercentile(r[fin], 2) - 1, np.nanpercentile(r[fin], 98) + 1)
    plt.tight_layout(); plt.show()
    print('%s: n=%d hitting terrain, median %.2f m, MAD %.2f m'
          % (title.split(':')[0], ok.sum(), np.nanmedian(r), np.nanmedian(np.abs(r))))

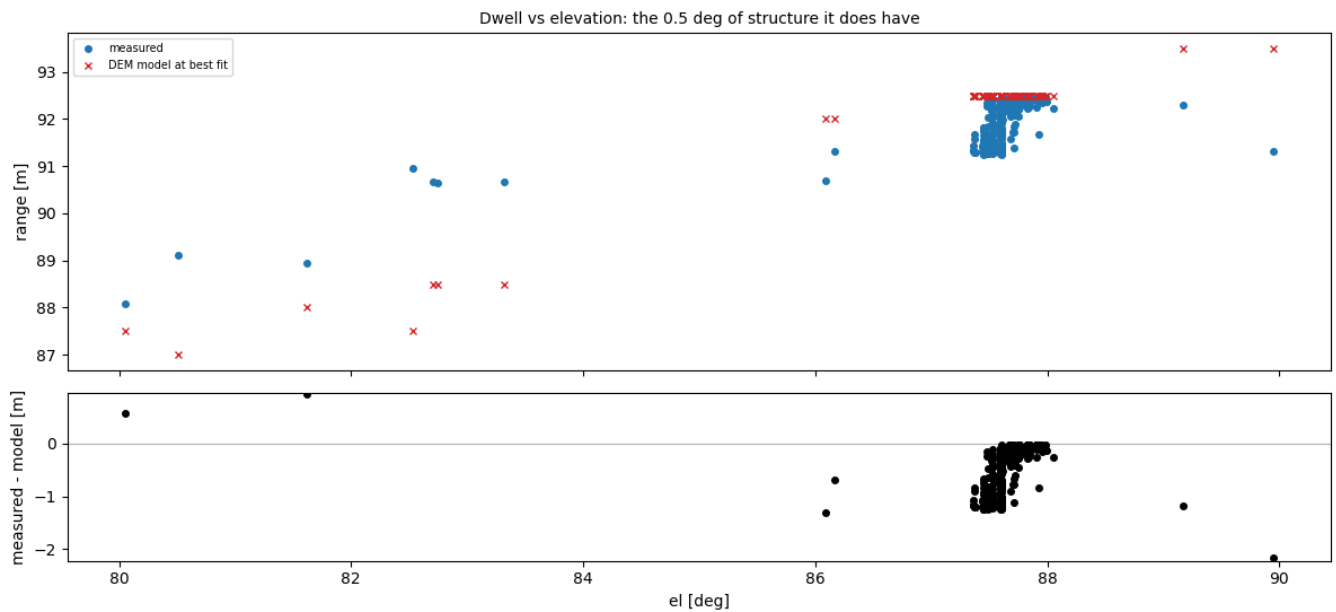
```

```
return r
```

```
mt = (t - t.min()) / 60.0
r_dw = overlay(dwell, 'plumb', mt, 'minutes from first return',
               'Dwell: measured vs DEM-predicted range')
r_dw_el = overlay(dwell, 'plumb', el, 'el [deg]',
                  'Dwell vs elevation: the 0.5 deg of structure it does have')
```

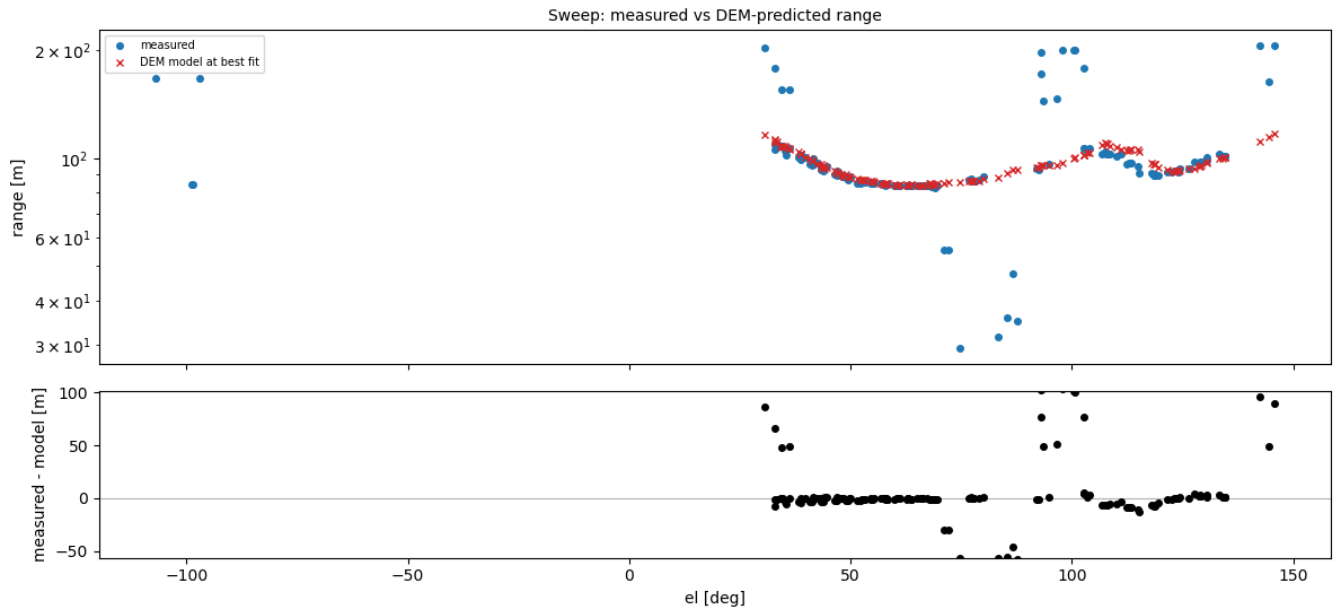


Dwell: n=488 hitting terrain, median -0.18 m, MAD 0.19 m



Dwell vs elevation: n=488 hitting terrain, median -0.18 m, MAD 0.19 m

```
r_sw = overlay(sweep, 'plumb', el, 'el [deg]',
               'Sweep: measured vs DEM-predicted range', logy=True)
```



Sweep: $n=147$ hitting terrain, median -0.42 m, MAD 1.26 m

The dwell overlay shows what a dwell looks like: the model is essentially a constant because the pointing is essentially a constant, and the residual is a flat offset with scatter. There is no profile to fit — which is exactly the point of §2.

The sweep overlay is the informative one. The model tracks the measured range over two decades of range across the `el` sweep, which is why the azimuth is recoverable at all; the visible failures are the near-tangential rays discussed in §7.

6. Refitting, at both elevation conventions

Scanning the azimuth offset `daz` and a height offset `du` against the masked sweep, scoring by median absolute residual — median because a fraction of the sweep rays graze the walls near-tangentially and are wild (§7).

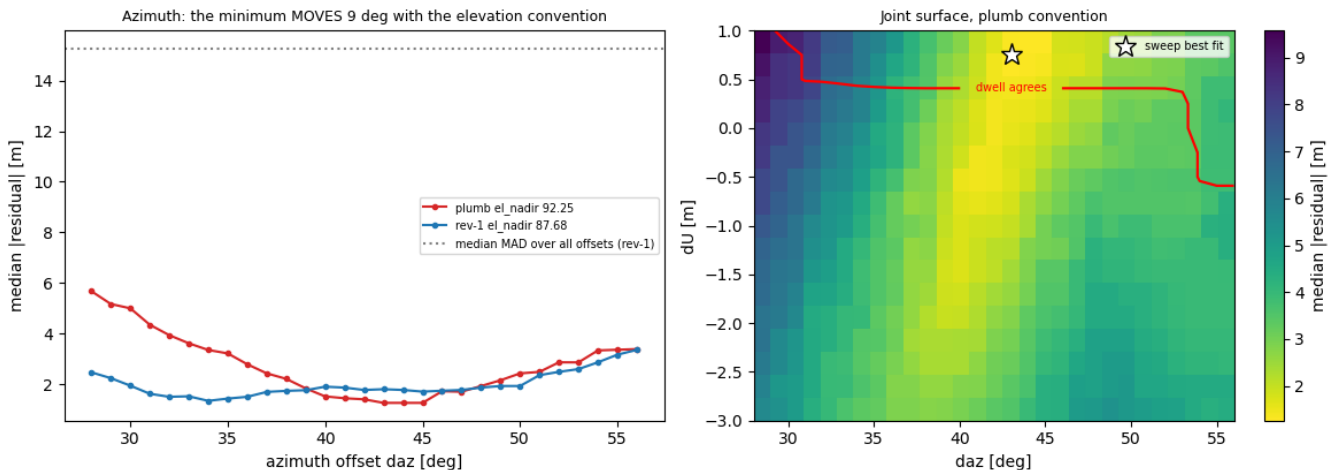
```
fig, axs = plt.subplots(1, 2, figsize=(12, 4.3))
for tag, col, lbl in (('plumb', 'tab:red', 'plumb el_nadir %.2f' % EL_PLUMB),
                     ('dwell_assumption', 'tab:blue', 'rev-1 el_nadir %.2f' % EL_DWELL)):
    m = Z['%s_mad' % tag]
    axs[0].plot(daz_grid, np.nanmin(m, axis=1), 'o-', ms=3, color=col, label=lbl)
    axs[0].axhline(15.27, color='0.5', ls=':', label='median MAD over all offsets (rev-1)')
    axs[0].set_xlabel('azimuth offset daz [deg]'); axs[0].set_ylabel('median |residual| [m]')
    axs[0].set_title('Azimuth: the minimum MOVES 9 deg with the elevation convention', fontsize=9)
    axs[0].legend(fontsize=7)

m = Z['plumb_mad']
im = axs[1].imshow(m.T, origin='lower', aspect='auto', cmap='viridis_r',
                  extent=(daz_grid[0], daz_grid[-1], du_grid[0], du_grid[-1]))
fig.colorbar(im, ax=axs[1], label='median |residual| [m]')
cs = axs[1].contour(daz_grid, du_grid, Z['plumb_plateau_resid'].T, levels=[0.],
                  colors='red', linewidths=1.6)
axs[1].clabel(cs, fmt='{0.: 'dwell agrees'}', fontsize=7)
b = MAIN['best_fit']
axs[1].plot(b['daz_deg'], b['du_m'], 'w*', ms=14, mec='k', label='sweep best fit')
axs[1].set_xlabel('daz [deg]'); axs[1].set_ylabel('du [m]')
axs[1].set_title('Joint surface, plumb convention', fontsize=9)
axs[1].legend(fontsize=7, loc='upper right')
plt.tight_layout(); plt.show()
```

```

print(f"{'convention':22s}{'el_nadir':>10}{'daz':>8}{'dU':>8}{'MAD':>8}")
print('-' * 56)
for tag, lbl in (('plumb', 'plumb-derived'), ('dwell_assumption', 'rev-1 assumption')):
    bb = S['by_el_nadir'][tag]['best_fit']
    print(f"{'by_el_nadir':22s}{bb['el_nadir_deg']:10.2f}{bb['daz_deg']:8.1f}{bb['dU_m']:8.2f}{bb['mad_m']:8.2f}")
print()
print('revision 1, contaminated by the 26 near-field samples:    daz 34.0    dU -1.25    MAD 1.57')

```



convention	el_nadir	daz	dU	MAD
plumb-derived	92.25	43.0	0.75	1.26
rev-1 assumption	87.68	34.0	-1.25	1.34

revision 1, contaminated by the 26 near-field samples: daz 34.0 dU -1.25 MAD 1.57

6.1 Disentangling the two changes

Two things changed at once — the near-field mask and the elevation registration — so attributing the improvement to either requires running the intermediate case. That is done below rather than assumed.

```

R1 = S['revision1_unmasked']
rows = [('rev-1: unmasked, el_nadir %.2f' % EL_DWELL, R1['daz_deg'], R1['dU_m'],
        R1['mad_m'], R1['daz_spread_deg'], R1['dU_spread_m'])]
for tag, lbl in (('dwell_assumption', 'masked, el_nadir %.2f' % EL_DWELL),
                ('plumb', 'masked, el_nadir %.2f (plumb)' % EL_PLUMB)):
    B = S['by_el_nadir'][tag]
    rb = B['robustness']
    dz = np.array([v['daz_deg'] for v in rb.values()])
    dv = np.array([v['dU_m'] for v in rb.values()])
    rows.append((lbl, B['best_fit']['daz_deg'], B['best_fit']['dU_m'],
                B['best_fit']['mad_m'], np.ptp(dz), np.ptp(dv)))
print(f"{'configuration':34s}{'daz':>7}{'dU':>7}{'MAD':>7}{'daz spread':>12}{'dU spread':>11}")
print('-' * 78)
for r_ in rows:
    print(f"{'r_[0]':34s}{r_[1]:7.0f}{r_[2]:+7.2f}{r_[3]:7.2f}{r_[4]:12.0f}{r_[5]:11.2f}")

```

configuration	daz	dU	MAD	daz spread	dU spread
rev-1: unmasked, el_nadir 87.68	34	-1.25	1.57	12	3.00
masked, el_nadir 87.68	34	-1.25	1.34	12	2.75
masked, el_nadir 92.25 (plumb)	43	+0.75	1.26	5	1.25

Read across those rows:

- **The mask improved fit quality and changed nothing else.** Same `daz` (34), same `dU` (−1.25), same 12° and ~3 m robustness spreads; MAD 1.57 → 1.34 m. The median-based score had already been immune to the 26 outliers.
- **The elevation registration is what actually tightens the measurement.** `daz` spread 12° → 5°, `dU` spread 2.75 m → 1.25 m, MAD 1.34 → 1.26 m.

That last line is an independent corroboration of §4. The plumb-line fit and the terrain fit are different data and different physics, and the terrain fit becomes more self-consistent at the elevation registration the plumb line independently prefers. The MAD improvement alone (6%, 151 points) would be weak; the halving of both robustness spreads is the stronger signal.

6.2 What can masquerade as height

```
print('dwell residual at the photogrammetric position (no fitting):')
for tag, lbl in (('plumb', 'plumb el_nadir'), ('dwell_assumption', 'rev-1 el_nadir')):
    print('    %-18s %+.3f m' % (lbl, S['by_el_nadir'][tag]['sensitivities']
['baseline_median_residual_m']))
print()
print('Sensitivity of that number (plumb convention), m per unit:')
sens = MAIN['sensitivities']
for k in ('antenna_E_plus_1m', 'antenna_N_plus_1m', 'antenna_U_plus_1m'):
    print('    %-22s %+.6.2f m per m' % (k, sens[k]['d_residual_per_m']))
for k in sorted(k for k in sens if k.startswith('el_nadir')):
    print('    %-22s %+.6.2f m' % (k, sens[k]['d_residual']))
for k in sorted(k for k in sens if k.startswith('daz')):
    print('    %-22s %+.6.2f m' % (k, sens[k]['d_residual']))
```

```
dwell residual at the photogrammetric position (no fitting):
plumb el_nadir      +1.315 m
rev-1 el_nadir      -1.468 m
```

```
Sensitivity of that number (plumb convention), m per unit:
antenna_E_plus_1m   +0.10 m per m
antenna_N_plus_1m   +0.99 m per m
antenna_U_plus_1m   -1.00 m per m
el_nadir_+1deg       +1.00 m
el_nadir_+2deg       +1.97 m
el_nadir_-1deg       -0.93 m
el_nadir_-2deg       -0.53 m
daz_+180deg         -6.92 m
daz_+40deg          -1.00 m
```

6.3 Was the east/north position ever allowed to move? No — and here is what happens when it is

Direct answer first: `dE` and `dN` were held fixed at the photogrammetric value in everything above. The ±2 m rows in the robustness table are fixed perturbations used to test stability, not a fit. Only `daz` and `dU` were ever optimised.

Aaron read the sweep overlay and observed that the broad first dip lines up well — which supports the elevation calibration — while the **second dip** near `el` 105–122 is displaced relative to the model, and asked whether antenna E/N could account for it. That is a structured shape mismatch, not scatter, so it is worth chasing.

The test: grid `dE`, `dN` over ± 8 m in 1 m steps, **re-optimising `daz` and `dU` at every cell**, scored on a shape set — sweep rays with measured range 80–130 m. That cut uses the measured values only, so it cannot be biased toward any model.

```
EN = S['en_exploration']
EZ = dict(np.load(RUN / 'en_scan.npz', allow_pickle=True))
G = EZ['G']; shp = EZ['shape'].astype(bool)
sel, saz, sd = EZ['el'], EZ['az'], EZ['d']
print(EN['note'])
print('shape set:', EN['shape_set'])
print('grid      : ', EN['grid'])
print()
print('Window medians of (measured - model), m:')
print(f"{'configuration':44s}{'W1':>8s}{'W2':>8s}{'W3':>8s}{'MAD':>8s}")
print('-' * 76)
for k, lbl in (('published', 'published: dE=0 dN=0, daz=43, dU=+0.75'),
              ('en_refit', 'E/N refit: dE=+1 dN=-6, daz=36, dU=-2.00')):
    c = EN['configs'][k]
    print(f"{'':44s}{c['W1']:8.2f}{c['W2']:8.2f}{c['W3']:8.2f}{c['MAD']:8.2f}")
w = EN['window_definitions']
print()
print('W1 = el %d-%d (first dip)   W2 = el %d-%d (SECOND dip)   W3 = el %d-%d'
      % (*w['W1'], *w['W2'], *w['W3']))
```

E/N was HELD FIXED in rev1/rev2 fits; the +/-2 m rows were perturbations, not a fit.
 shape set: sweep rays with measured range 80–130 m (data-only cut), n=127
 grid : dE,dN in [-8,8] m step 1; daz and dU re-optimised at every cell

Window medians of (measured - model), m:				
configuration	W1	W2	W3	MAD
published: dE=0 dN=0, daz=43, dU=+0.75	1.07	-6.44	0.93	1.03
E/N refit: dE=+1 dN=-6, daz=36, dU=-2.00	1.03	-3.90	-0.15	0.79

W1 = el 88–105 (first dip) W2 = el 105–122 (SECOND dip) W3 = el 122–140

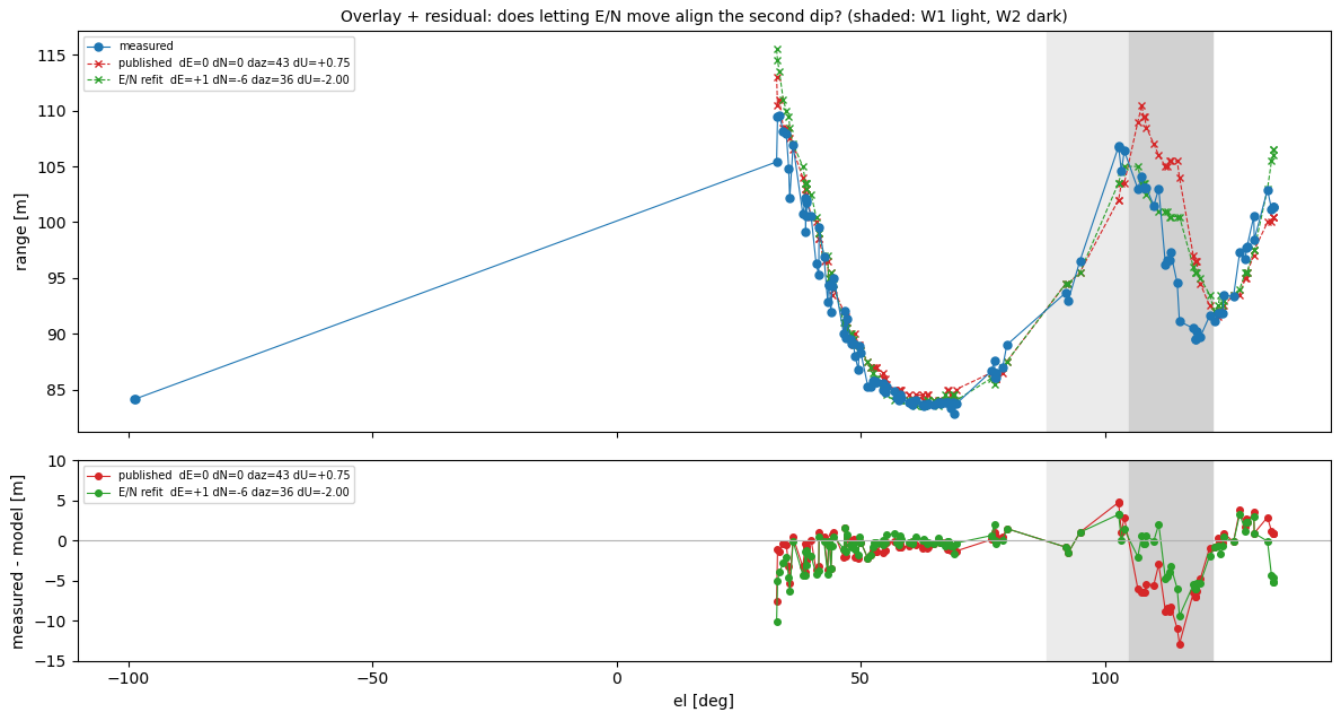
```
import marjum_lidar_constraint as MLC
from eigsep_terrain.marjum_dem import MarjumDEM as DEM2
from marjum_bundle import working_grid as _wg
dem2 = _wg(DEM2(cache_file='marjum_dem_sw.npz'))

k = np.argsort(sel); msk = shp[k]; xx = sel[k][msk]; yy = sd[k][msk]
fig, axs = plt.subplots(2, 1, figsize=(12, 6.5), sharex=True,
                        gridspec_kw=dict(height_ratios=[2, 1]))
axs[0].plot(xx, yy, 'o-', ms=5, lw=.8, color='tab:blue', label='measured', zorder=3)
for key, col, lbl in (('published', 'tab:red', 'published dE=0 dN=0 daz=43 dU=+0.75'),
                     ('en_refit', 'tab:green', 'E/N refit dE=+1 dN=-6 daz=36 dU=-2.00')):
    c = EN['configs'][key]
    pr = MLC.march(dem2, np.array(ANT) + np.array([c['dE'], c['dN'], c['dU']]),
                  sel, saz + c['daz'], EL_PLUMB)
    axs[0].plot(xx, pr[k][msk], 'x--', ms=5, lw=.8, color=col, label=lbl)
    axs[1].plot(xx, (sd - pr)[k][msk], 'o-', ms=4, lw=.8, color=col, label=lbl)
axs[1].axhline(0, color='0.7', lw=.8)
for a in axs:
    a.axvspan(*w['W1'], color='0.92', zorder=0)
```

```

a.axvspan(*w['W2'], color='0.82', zorder=0)
axs[0].set_ylabel('range [m]'); axs[0].legend(fontsize=7, loc='upper left')
axs[0].set_title('Overlay + residual: does letting E/N move align the second dip? '
                '(shaded: W1 light, W2 dark)', fontsize=10)
axs[1].set_ylabel('measured - model [m]'); axs[1].set_xlabel('el [deg]')
axs[1].set_ylim(-15, 10); axs[1].legend(fontsize=7)
plt.tight_layout(); plt.show()

```



It genuinely helps. In the second-dip window the median residual improves from **−6.44 m** to **−3.90 m**, the overall shape-set MAD from 0.94 to 0.79 m, and W1 and W3 do not degrade. On the rising edge near `el` 108 the improvement is large — the published model sits ~6.4 m long there and the refit ~0.4 m. The preferred shift is about **6 m south**, `dE` $\approx$ 0.

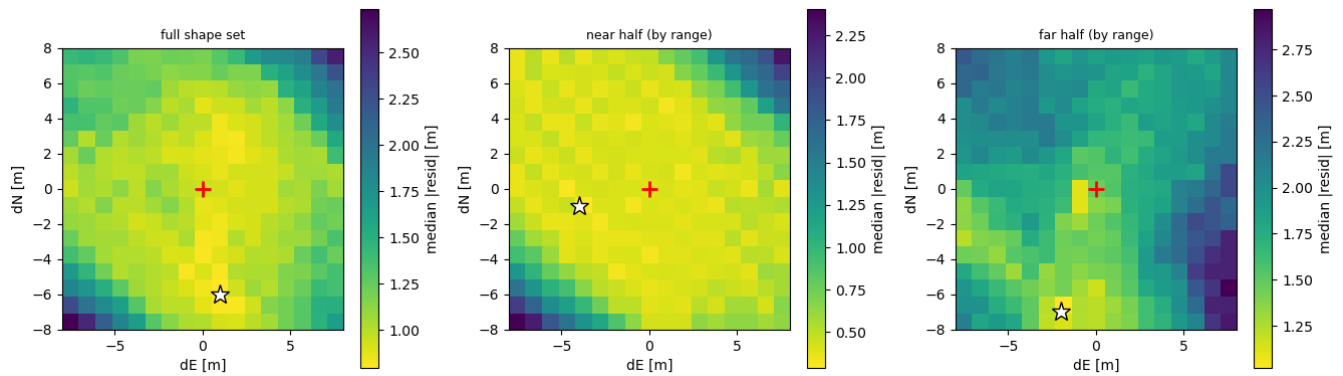
But it should not be quoted as a position constraint, for three reasons.

```

fig, axs = plt.subplots(1, 3, figsize=(13.5, 4.0))
for ax, key, ttl in ((axs[0], 'R', 'full shape set'), (axs[1], 'Rn', 'near half (by range)'),
                    (axs[2], 'Rf', 'far half (by range)')):
    X = EZ[key]
    im = ax.imshow(X.T, origin='lower', cmap='viridis_r',
                  extent=(G[0], G[-1], G[0], G[-1]))
    fig.colorbar(im, ax=ax, label='median |resid| [m]')
    i, j = np.unravel_index(np.nanargmin(X), X.shape)
    ax.plot(G[i], G[j], 'w*', ms=14, mec='k')
    ax.plot(0, 0, 'r+', ms=12, mew=2)
    ax.set_xlabel('dE [m]'); ax.set_ylabel('dN [m]'); ax.set_title(ttl, fontsize=9)
plt.tight_layout(); plt.show()

print(f"{'subset':22s}{'dE':>7f}{'dN':>7f}{'best':>9f}{'at origin':>11f}{'flat frac':>11f}")
print('-' * 67)
for k, v in EN['subsets'].items():
    print(f"{k:22s}{v['dE']:7.0f}{v['dN']:7.0f}{v['val']:9.3f}{v['at_origin']:11.3f}"
          f"{v['flat_fraction']:11.2f}")

```



subset	dE	dN	best	at origin	flat frac
full_shape	1	-6	0.793	0.940	0.07
half1_time	-6	1	0.382	0.538	0.10
half2_time	1	-5	1.030	1.520	0.02
near_half_range	-4	-1	0.287	0.380	0.01
far_half_range	-2	-7	1.018	1.463	0.01

1. The preferred offset does not reproduce across subsets. Four subsets that localise at all give $(dE, dN) = (+1, -6), (+1, -5), (-4, -1)$ and $(-2, -7)$ — a scatter of ~ 5 m in E and ~ 6 m in N, as large as the shift itself. The fifth (first half by time) has **no constraining power**: cells within 10% of its minimum are spread over the whole grid, and it scores 0.462 at the full fit's optimum against 0.382 at its own — so it does not contradict the others, it simply has no opinion. I initially read that as a contradiction; it is not, but neither is it support.

2. A rigid translation cannot make the second dip deep enough. The measured minimum near e_l 118 reaches **89.5 m**; the published model bottoms at 96.5 m and the E/N refit at 95.5 m. Moving the antenna displaces a dip, it does not deepen it. The leading edge of the feature is fixable by geometry and the depth is not, which points at the **DEM** — an unresolved feature in the canyon wall along this bearing — rather than at antenna position.

3. The magnitude is in conflict with the photogrammetry. A 6 m southward shift is **3.5× the ± 1.7 m antenna bracket**. Either the photogrammetry is badly wrong in a direction nothing else has suggested, or the LIDAR fit is using E/N as a free parameter to absorb terrain error. Given points 1 and 2, the second is much more likely — and this is the ordinary failure mode of adding parameters to absorb a structured residual whose cause is in the model, not the geometry.

So: Aaron's reading of the plot was right, and the follow-through does not support the position interpretation. The second dip is a real structured mismatch; E/N can absorb about 40% of it; the remainder is a depth error a translation cannot produce. The honest conclusion is that this is DEM error, and that the LIDAR data still constrains only daz — now with the added knowledge that daz and dN trade against each other (the refit moved daz from 43° to 36° when dN went to -6 m), which **widens** the azimuth uncertainty rather than narrowing it.

7. Robustness — which parameter survives resampling?

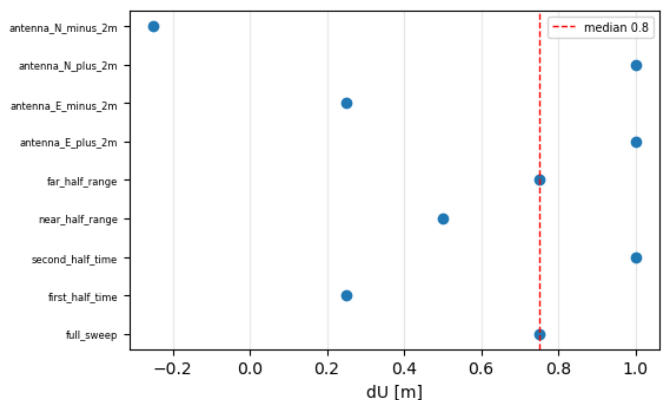
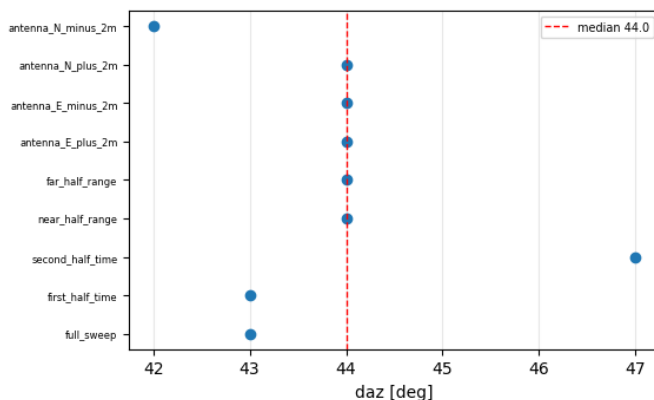
A fitted parameter that is real reappears when the data are split. This decides what may be quoted.

```
rob = MAIN['robustness']
order = ['full_sweep', 'first_half_time', 'second_half_time',
        'near_half_range', 'far_half_range',
        'antenna_E_plus_2m', 'antenna_E_minus_2m',
        'antenna_N_plus_2m', 'antenna_N_minus_2m']
order = [k for k in order if k in rob]
print(f"{subset / perturbation':26s}{ 'n':>5}{ 'daz':>8}{ 'dU':>8}{ 'MAD':>8}")
print('-' * 55)
for k in order:
    r = rob[k]
    print(f"{k:26s}{r['n']:5d}{r['daz_deg']:8.1f}{r['dU_m']:8.2f}{r['mad_m']:8.2f}")
dz = np.array([rob[k]['daz_deg'] for k in order])
du = np.array([rob[k]['dU_m'] for k in order])
print()
print('daz : %.0f..%.0f deg (spread %.0f)' % (dz.min(), dz.max(), np.ptp(dz)))
print('dU : %+.2f..%+.2f m (spread %.2f)' % (du.min(), du.max(), np.ptp(du)))

fig, axs = plt.subplots(1, 2, figsize=(11.5, 3.6))
y = np.arange(len(order))
for ax, v, lbl in ((axs[0], dz, 'daz [deg]'), (axs[1], du, 'dU [m]')):
    ax.plot(y, v, 'o', color='tab:blue')
    ax.axvline(np.median(v), color='r', ls='--', lw=1, label='median %.1f' % np.median(v))
    ax.set_yticks(y); ax.set_yticklabels(order, fontsize=6)
    ax.set_xlabel(lbl); ax.legend(fontsize=7); ax.grid(alpha=.3, axis='x')
plt.tight_layout(); plt.show()
```

subset / perturbation	n	daz	dU	MAD
full_sweep	151	43.0	0.75	1.26
first_half_time	75	43.0	0.25	0.65
second_half_time	76	47.0	1.00	2.40
near_half_range	75	44.0	0.50	0.44
far_half_range	76	44.0	0.75	2.48
antenna_E_plus_2m	151	44.0	1.00	1.21
antenna_E_minus_2m	151	44.0	0.25	1.50
antenna_N_plus_2m	151	44.0	1.00	1.26
antenna_N_minus_2m	151	42.0	-0.25	1.18

daz : 42..47 deg (spread 5)
dU : -0.25..+1.00 m (spread 1.25)



```

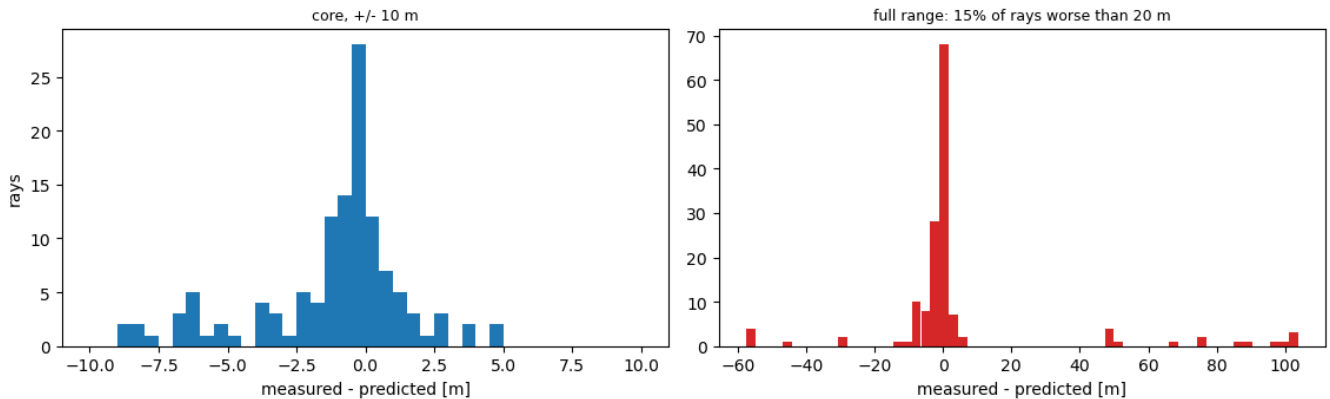
r = MAIN['residuals_at_optimum']
for k, v in r.items():
    print('%-18s %s' % (k, ('%d' % v) if k == 'n' else ('%.3f' % v)))
resid = Z['plumb_resid']
fig, axs = plt.subplots(1, 2, figsize=(11.5, 3.6))
axs[0].hist(resid, bins=np.arange(-10, 10.5, .5), color='tab:blue')
axs[0].set_xlabel('measured - predicted [m]'); axs[0].set_ylabel('rays')
axs[0].set_title('core, +/- 10 m', fontsize=9)
axs[1].hist(resid, bins=60, color='tab:red')
axs[1].set_xlabel('measured - predicted [m]')
axs[1].set_title('full range: %.0f%% of rays worse than 20 m'
                % (100 * r['frac_beyond_20m']), fontsize=9)
plt.tight_layout(); plt.show()

```

```

n                147
median_m         -0.423
mad_m            1.260
sd_m             27.331
frac_within_1m   0.408
frac_within_2m   0.578
frac_within_5m   0.728
frac_beyond_20m  0.150

```



The residual is a **mixture, not a distribution** — the standard deviation and the median absolute residual disagree by more than an order of magnitude. The tail is expected and its causes are identifiable: a ray grazing a canyon wall near-tangentially changes range by tens of metres for a fraction of a degree of pointing error; the DEM is 1 m-quantised and carries no vegetation, talus or suspension structure; and the antenna sways during the sweep. A least-squares treatment of this data would be wrong.

8. Where the LIDAR actually looked

```

from eigsep_terrain.marjum_dem import MarjumDEM as DEM
from marjum_bundle import working_grid
from terrain_hillshade_cmap import plot_terrain_hillshade

dem = working_grid(DEM(cache_file='marjum_dem_sw.npz'))
se, sn = dem.working_shift
dem_raw = DEM(cache_file='marjum_dem_sw.npz')

pts = Z['plumb_ground_pts']
fin = np.isfinite(pts[:, 0])
pts = pts[fin]

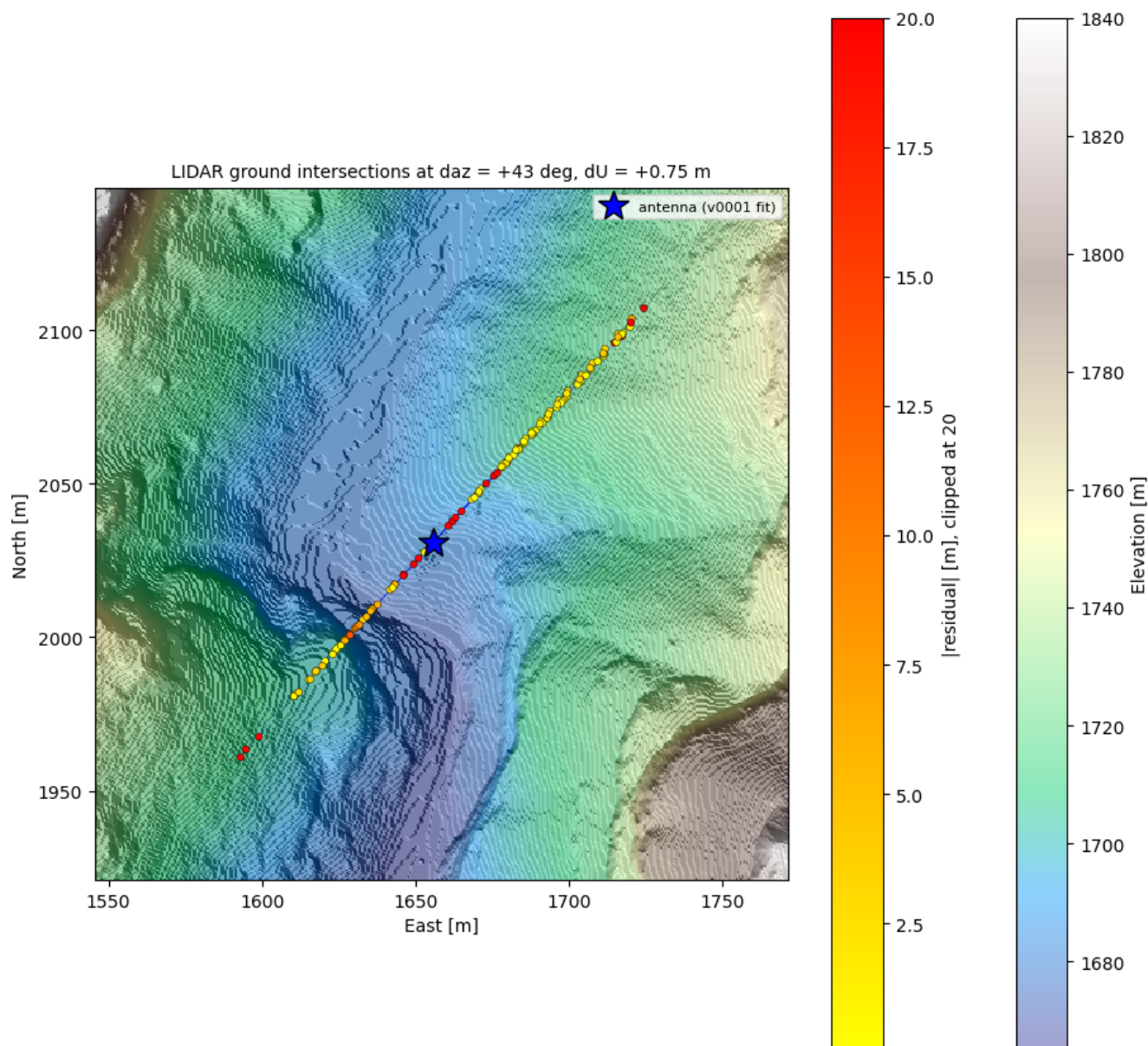
```

```

lo = np.minimum(pts[:, :2].min(0), ANT[:2]) - 40
hi = np.maximum(pts[:, :2].max(0), ANT[:2]) + 40
e0, n0 = (lo + hi) / 2
rad = float(max(hi - lo) / 2)
E, N, U = dem_raw.get_tile(erng_m=(e0 - se - rad, e0 - se + rad),
                           nrng_m=(n0 - sn - rad, n0 - sn + rad), mesh=False)
E, N = E + se, N + sn

fig, ax = plt.subplots(figsize=(9.5, 8.5))
im = plot_terrain_hillshade(ax, U, res=float(dem.res), extent=(E[0], E[-1], N[0], N[-1]))
fig.colorbar(im, ax=ax, label='Elevation [m]')
sc = ax.scatter(pts[:, 0], pts[:, 1], c=np.clip(np.abs(resid[:len(pts)]), 0, 20), s=16,
               cmap='autumn_r', edgecolors='k', linewidths=.3, zorder=3)
fig.colorbar(sc, ax=ax, label='|residual| [m], clipped at 20')
ax.plot(ANT[0], ANT[1], 'b*', ms=18, mec='k', zorder=4, label='antenna (v0001 fit)')
for p in pts[:4]:
    ax.plot([ANT[0], p[0]], [ANT[1], p[1]], color='b', lw=.25, alpha=.25, zorder=2)
ax.set_xlabel('East [m]'); ax.set_ylabel('North [m]'); ax.set_aspect('equal')
ax.set_title('LIDAR ground intersections at daz = %+.0f deg, dU = %+.2f m'
             % (MAIN['best_fit']['daz_deg'], MAIN['best_fit']['dU_m']), fontsize=10)
ax.legend(fontsize=8, loc='best')
plt.tight_layout(); plt.show()

```



The footprint is a line, not a fan. Every good return sits within **1.3° of azimuth**, so the whole dataset lies in a single vertical plane through the antenna. The fit constrains the **orientation** of that plane — which is exactly `daz`, and well — and position within it, partially; it constrains **almost nothing perpendicular to the plane**. Any statement about antenna horizontal position from this data is one-dimensional. That is also why the ± 2 m E and N perturbations in §7 barely change the fit quality: one of those directions is nearly unconstrained by construction.

9. Read-out

1. **The glitch is the support frame, and it is now masked.** 26 of 665 returns, `d` < 10 m, separated from terrain by a 23 m gap. All 26 were in revision 1's sweep fit — 15% maximally-wrong data. Flagged plainly rather than quietly refitted.
2. **The frame gives the first real elevation registration.** LIDAR nadir at `el` = **92.25°, $\sim 1\sigma$ 91.0–93.8° — +4.6°** from the value revision 1 assumed, which was never a measurement of nadir but the dwell's parked elevation. The plumb-line interpretation comes from the hardware; the fit supplies only the direction.
3. **Elevation registration is the dominant systematic**, at ~ 1 m of apparent antenna height per degree. A +4.6° shift is worth ~ 4.6 m. Every fit is therefore reported at both conventions.
4. **The azimuth registration moved, and its revision-1 error bar was too small.** `daz` goes from 34° to **43°** when the elevation registration is corrected — a 9° shift, against the $\pm 6^\circ$ I quoted in revision 1. The estimate is better now, not worse: at the plumb convention the subset spread is 5° rather than 12°. But `daz` and `el_nadir` are coupled at roughly 2° per degree, so the $\pm 1.4^\circ$ on `el_nadir` contributes $\pm 2.8^\circ$, and the honest number is `daz` $\approx$ **43° $\pm$ 4°**. Revision 1's $\pm 6^\circ$ did not cover the true value.
5. **The LIDAR still does not constrain antenna height** — and the two conventions make that unusually clear, since `du` lands at -1.25 m under one and $+0.75$ m under the other. It straddles zero. On top of that the elevation registration's own $\pm 1.4^\circ$ is worth $\sim \pm 1.4$ m, before the DEM's 1 m quantisation and the contribution from horizontal position on a 22.9° slope.
6. **All of it lies in one vertical plane.** Nothing is constrained perpendicular to it. Any future LIDAR use for horizontal position needs returns at a second azimuth; this dataset cannot supply one.
7. **East/north was held fixed, and letting it move does not rescue the structured residual** (§6.3). It absorbs roughly 40% of the second-dip mismatch and improves the shape-set MAD from 0.94 to 0.79 m, but the preferred offset scatters ~ 6 m across subsets, exceeds the photogrammetric bracket by 3.5 $\times$, and cannot reproduce the depth of the measured dip at all. The residue is most likely DEM error along that bearing.
8. `daz` and `dn` **trade against each other**, which is the one firm consequence of §6.3: allowing a 6 m southward shift moved the azimuth from 43° to 36°. That **widens** the azimuth uncertainty. Quoting `daz` = 43° $\pm$ 4° is only valid conditional on the

photogrammetric E/N; released from that, the supportable statement is roughly **36-47°**.

Decision requested

- **Q9 — Hand the azimuth registration to the pointing table?** It is the product the table lists as an open dependency against `geometer`. It is coarse next to the table's $\sim 0.4^\circ$ relative precision, so my recommendation is to hand it over explicitly flagged as a coarse absolute anchor. Data-archivist's product, Aaron's call, needs routing.
- **Q12 (new) — Should the elevation registration go to the pointing table too?** `el_nadir` = $92.25^\circ \pm 1.4^\circ$ is a statement about the instrument, not about this analysis, and anyone using `el_deg` for beam work needs it. It also bears on the boresight branch, since it shifts where "boresight vertical" sits by 4.6° . I did not fold it into the README in §1's correction because it postdates that edit — it should be a second, separate correction if approved.
- **Q10 — Is a sub-metre LIDAR height constraint worth buying?** It needs a better DEM under the antenna, `el_nadir` to $\leq 0.5^\circ$, and the lever arm measured. That is a field/acquisition ask, not an analysis ask. I would not spend more analysis on the existing data.
- **Q11 — Priority of the transmitter power-peak test.** Its remaining unique value is the boresight branch; it can now be cross-checked against the azimuth number rather than having to supply it.

No re-fit, reparameterisation, release or new run was started. The photogrammetric antenna position is unchanged.